Vežba: zadatak klasifikacije teksta (vektori reči, SVM i SMOTE)

U fajlu “Tweets.csv”[1] su dati tvitovi putnika upućeni američkim avio-kompanijama (februar 2015). Svaki red je jedan tvit. Fajl sadrži sledeće kolone (varijable):

- **tweet_id** - identifikator tvita
- **airline_sentiment** - ručno označen sentiment tvita (negative, neutral, positive)
- **airline_sentiment_confidence** - koliko su se označivači složili oko sentimenta (0 do 1)
- **negativereason** - razlog nezadovoljstva (npr. Late Flight, Lost Luggage); popunjeno samo za negativne tvitove
- **negativereason_confidence** - pouzdanost oznake razloga
- **airline** - avio-kompanija kojoj je tvit upućen (6 kompanija)
- **airline_sentiment_gold, negativereason_gold** - kontrolne oznake, skoro uvek prazne
- **name** - korisničko ime autora tvita
- **retweet_count** - broj ponovnih objava tvita
- **text** - tekst tvita
- **tweet_coord, tweet_created, tweet_location, user_timezone** - koordinate, vreme objave, lokacija i vremenska zona korisnika

Cilj je napraviti model koji automatski prepoznaje sentiment tvita, kako bi služba za korisnike avio-kompanije brže uočila nezadovoljne putnike. Zbog trajanja računanja vektora reči i kros-validacije, nakon čišćenja podataka iz skupa nasumično izdvojiti **4000 tvitova** (*data.sample(n=4000, random_state=1)*) i dalje raditi samo sa njima.

Koristeći opisane podatke iz navedenog fajla, potrebno je uraditi sledeće:

- Učitati podatke i ispitati ih (tipovi kolona, nedostajuće vrednosti, duplikati). Izbaciti tvitove kod kojih nedostaje tekst, kao i tvitove sa duplikatom teksta. Obratiti pažnju da *dropna()* bez argumenata ovde ne sme da se koristi i obrazložiti zašto.
- Polazeći od kolone **airline_sentiment** napraviti novu varijablu **sentiment** sa tri moguće vrednosti, primenjujući sledeće mapiranje:
    - negative se mapira u “negativan”,
    - neutral se mapira u “neutralan”,
    - positive se mapira u “pozitivan”.

  Varijabla sentiment bi trebalo da posluži kao izlazna varijabla za klasifikaciju u ovom zadatku. Prikazati broj i udeo tvitova po klasama i prokomentarisati šta neizbalansiranost klasa znači za kasniju evaluaciju i koja klasa će verovatno biti najslabija.
- Proceniti da li bi još neku od varijabli iz polaznog dataset-a trebalo primeniti za kreiranje modela. Za svaku kolonu osim text ukratko obrazložiti odluku (uključiti / ne uključiti). Za kolonu retweet_count odluku potkrepiti grafikom raspodele po klasama (kdeplot, strana 3), a za kolone airline, tweet_location i user_timezone brojem različitih

vrednosti i brojem nedostajućih vrednosti. Posebno razmotriti kolone negativereason i airline_sentiment_confidence: da li bi te vrednosti bile poznate u trenutku kada model treba da klasifikuje novi tvit (curenje podataka)? Kao ulaz u model zadržati samo kolonu text.

- Transformisati kolonu text u numerički oblik primenom vektora reči iz spaCy modela **en_core_web_md** (model en_core_web_sm nema vektore reči). Pre računanja vektora tekst očistiti uz pomoć spaCy biblioteke (lematizacija, uklanjanje interpunkcije, razmaka, brojeva, URL-ova, e-mail adresa, pominjanja korisnika koja počinju sa @ i stop reči, pri čemu reči “not” i “no” treba zadržati). Svaki tvit predstaviti kao prosek vektora preostalih reči (vektor dužine 300). Funkciju koja pretvara listu tekstova u matricu vektora uvesti u Pipeline pomoću klase *FunctionTransformer*. U komentaru objasniti zašto ova transformacija ne uči ništa iz podataka i šta to znači za podelu na trening i test skup.
- Podeliti podatke na trening i test skup u odnosu 80:20, tako da udeo klasa bude isti u oba skupa. Izlaznu varijablu kodirati brojevima pomoću *pd.factorize*, a nazive klasa sačuvati za prikaz izveštaja.
- Kreirati prvi klasifikacioni model primenom algoritma Support Vector Machines (klasa SVC, *kernel='linear', C=1, decision_function_shape='ovo'*), uz korak SMOTE (oversampling manjinskih klasa) između vektorizacije i klasifikatora. Vektorizaciju, SMOTE i model objediniti u jedan Pipeline iz biblioteke imblearn. Ukratko objasniti kako SMOTE kreira nove primere, zašto se primenjuje samo nad trening podacima, a ne pre podele na trening i test skup, i zašto mora da stoji posle vektorizacije. U komentaru navesti šta predstavlja hiperparametar C i drugi način da se SVM-u ukaže na neizbalansiranost klasa, bez kreiranja novih primera.
- Napraviti predikcije primenom kreiranog modela nad test skupom i izračunati standardne evaluacione metrike (preciznost, odziv, F1) kako za svaku klasu pojedinačno, tako i sveukupno tj na nivou svih klasa, kao i balanced accuracy. Prikazati matricu konfuzije. Interpretirati dobijene vrednosti metrika u kontekstu zadatka: koja klasa je najslabija, sa kojom klasom se najčešće meša (na osnovu matrice konfuzije) i šta to praktično znači za službu za korisnike. Posebno protumačiti razliku između macro i weighted F1.
- Kreirati drugi klasifikacioni model kroz postupak podešavanja hiperparametara. U komentaru navesti koje biste hiperparametre podešavali i na koji način tj koristeći koje opsege / liste vrednosti ako ne biste bili vremenski ograničeni (C, kernel, gamma, k_neighbors kod SMOTE-a). Zatim, izabrati dva hiperparametra klasifikatora, C i kernel (vektori reči nemaju hiperparametre transformacije), i izvršiti podešavanje njihovih vrednosti kroz postupak (5-fold) kros-validacije, koristeći macro F1 kao metriku. Za C isprobati 3 vrednosti na logaritamskoj skali, a za kernel vrednosti linear i rbf.
- Izvršiti predikcije primenom podešenog modela i izračunati standardne evaluacione metrike (preciznost, odziv, F1) kako za svaku klasu pojedinačno, tako i sveukupno tj na nivou svih klasa. Uporediti najbolji CV macro F1 (best_score_) sa macro F1 na test skupu i prokomentarisati da li model overfituje.
- Uporediti i prokomentarisati rezultate dobijene osnovnim i podešenim modelom, koristeći datu funkciju get_model_metrics (strana 3). Da li je podešavanje poboljšalo model i za koju klasu? Ako poboljšanja nema ili je zanemarljivo, to i napisati. Koji model biste preporučili službi za korisnike i zašto?

[1] Podaci: Kaggle, “Twitter US Airline Sentiment” (Figure Eight / CrowdFlower), https://www.kaggle.com/

datasets/crowdflower/twitter-airline-sentiment. Preuzeti fajl Tweets.csv i kopirati ga u folder data pored sveske.

Dati kod

Sledeću funkciju dobijate uz zadatak i možete je koristiti bez izmena:

```
def get_model_metrics(y_true: np.ndarray, y_pred: np.ndarray, model_name: str):
    report = metrics.classification_report(y_true, y_pred, output_dict=True)
    macro_avg = report['macro avg']
    weighted_avg = report['weighted avg']
    return {
        'Model': model_name,
        'Macro Precision': macro_avg['precision'],
        'Macro Recall': macro_avg['recall'],
        'Macro F1-Score': macro_avg['f1-score'],
        'Weighted Precision': weighted_avg['precision'],
        'Weighted Recall': weighted_avg['recall'],
        'Weighted F1-Score': weighted_avg['f1-score'],
        'Balanced Accuracy': metrics.balanced_accuracy_score(y_true, y_pred)
    }
```

Primer koda za grafik raspodele numeričke kolone po klasama:

```
plt.figure(figsize=(10, 6))
sb.kdeplot(data=data, x='retweet_count', hue='sentiment',
           fill=True, common_norm=False, alpha=0.5, linewidth=2)
plt.title('Distribucija broja retvitova po sentimentu')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

Importi potrebni za vektore reči i SMOTE:

```
from sklearn.preprocessing import FunctionTransformer
from sklearn.svm import SVC
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline    # zamenjuje sklearn.pipeline.Pipeline
```

Napomene za izradu

- Rešenje predati kao Jupyter svesku (.ipynb). Svaki korak zadatka započeti markdown ćelijom sa kratkim opisom, a tumačenja i obrazloženja pisati u markdown ćelijama, ne samo u komentarima koda.
- Svuda gde postoji parametar random_state (uzorkovanje, podela podataka, SMOTE, SVC) koristiti vrednost 1.
- spaCy model učitati sa *spacy.load('en_core_web_md', disable=['parser', 'ner'])*; parser i NER nisu potrebni, a usporavaju obradu.
- Pipeline iz sklearn biblioteke ne dozvoljava korak koji menja broj redova (SMOTE), pa se mora koristiti Pipeline iz imblearn biblioteke.
- Ime hiperparametra u mreži za podešavanje sastoji se od imena koraka i imena hiperparametra razdvojenih sa dve donje crte (npr. classifier__C).
- Podešavanje hiperparametara raditi isključivo nad trening skupom; test skup koristiti samo za konačnu ocenu.
- Ako se ceo Pipeline prosledi u GridSearchCV, vektori reči se ponovo računaju u svakom foldu za svaku kombinaciju, što traje dugo. Dozvoljeno je vektore trening skupa izračunati jednom, pre GridSearchCV, i podešavati Pipeline koji sadrži samo

SMOTE i SVC. U tom slučaju najbolji model dobija vektore test skupa (*best_model.predict(vectorizer.transform(X_test))*). Obrazložiti zašto ovde to ne dovodi do curenja podataka.

- Ne povećavati mrežu preko 6 kombinacija.
- Brojevi sami po sebi nisu dovoljni: svaka metrika treba da bude protumačena u kontekstu tvitova putnika i službe za korisnike.